

WEEK 2: PRODUCT AND CATEGORY MANAGEMENT SYSTEM

1. Requirement Analysis

Product and Category Management

The e-commerce system needs to maintain information about products and their categories. Each product belongs to a particular category. The system stores important product details such as product name, price, and stock quantity.

The main objective of this module is to create and manage the **Category** and **Product** tables and establish a relationship between them.

Relationship

One category can contain many products. Therefore, the relationship between the Category and Product tables is **One-to-Many (1 → *)**.

- **Category_ID** is the Primary Key in the Category table.
- **Product_ID** is the Primary Key in the Product table.
- **Category_ID** acts as the Foreign Key in the Product table.

2. Category Table Creation

A Category table was created to store category information. The table contains **Category ID, Category Name, and Description**.

```
SQL> CREATE TABLE Category(  
2     Category_ID VARCHAR2(7),  
3     Category_Name VARCHAR2(20),  
4     Description VARCHAR2(50)  
5 );
```

Table created.

3. Product Table Creation

A Product table was created to store product information. It contains **Product ID, Product Name, Category ID, Price, and Stock Quantity**.

```
SQL> CREATE TABLE Product(  
 2     Product_ID VARCHAR2(7),  
 3     Product_Name VARCHAR2(20),  
 4     Category_ID VARCHAR2(7),  
 5     Price NUMBER(9,2),  
 6     Stock_Quantity NUMBER  
 7 );
```

Table created.

4. Primary Key Constraints

Primary key constraints were added to both tables.

- Category_ID was defined as the Primary Key of the Category table.
- Product_ID was defined as the Primary Key of the Product table.

```
SQL> ALTER TABLE Category  
 2 ADD CONSTRAINT PK_Category PRIMARY KEY (Category_ID);
```

Table altered.

```
SQL> ALTER TABLE Product  
 2 ADD CONSTRAINT PK_Product PRIMARY KEY (Product_ID);
```

Table altered.

5. Foreign Key Relationship

A foreign key relationship was established between the Product and Category tables. The Category_ID in the Product table references the Category_ID in the Category table.

This ensures that every product is associated with an existing category.

```
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT FK_Product_Category
  3  FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID);

Table altered.
```

6. NOT NULL Constraints

NOT NULL constraints were applied to the category name and product name. This ensures that these fields cannot contain empty values.

```
SQL> ALTER TABLE Category
  2  MODIFY Category_Name VARCHAR2(100) NOT NULL;

Table altered.

SQL> ALTER TABLE Product
  2  MODIFY Product_Name VARCHAR2(150) NOT NULL;

Table altered.
```

7. CHECK Constraints

CHECK constraints were added to validate product values.

- Product price must be greater than zero.
- Stock quantity must be zero or greater.

These constraints help maintain valid product data.

```
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT CHK_Price CHECK (Price > 0);

Table altered.

SQL> ALTER TABLE Product
  2  ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);

Table altered.
```

8. UNIQUE Constraint

A UNIQUE constraint was applied to the Category Name so that duplicate category names cannot be entered.

```
SQL> ALTER TABLE Category
  2  ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);

Table altered.
```

9. Insert Category Values

Four category records were inserted into the Category table.

Category ID	Category Name	Description
Cat0011	Mobiles	Smartphones and mobile devices
Cat0012	Footwear	Shoes and sandals
Cat0013	Furniture	Home furniture products
Cat0014	Stationery	School and office supplies

```
SQL> INSERT INTO Category
  2  VALUES(
  3      'Cat0011',
  4      'Mobiles',
  5      'Smartphones and mobile devices'
  6  );

1 row created.
```

10. Insert Product Values

Three product records were prepared for the Product table.

Product ID	Product Name	Category ID	Price	Stock Quantity
P2001	Smartphone	Cat0011	₹28,500	15
P2002	Running Shoes	Cat0012	₹3,499	25
P2003	Sofa	Cat0013	₹42,000	8

```
SQL> INSERT INTO Product
2 VALUES(
3      'P2001',
4      'Smartphone',
5      'Cat0011',
6      28500,
7      15
8  );

1 row created.
```

11. Display Category and Product Records

The inserted records were displayed using the SELECT operation to verify that the data was successfully stored in the tables.

```
SQL> SELECT * FROM Category;
```

```
CATEGOR
```

```
-----
```

```
CATEGORY_NAME
```

```
-----
```

```
DESCRIPTION
```

```
-----
```

```
Cat0011
```

```
Mobiles
```

```
Smartphones and mobile devices
```

```
Cat0012
```

```
Footwear
```

```
Shoes and sandals
```

```
CATEGOR
```

```
-----
```

```
CATEGORY_NAME
```

```
-----
```

```
DESCRIPTION
```

```
-----
```

```
Cat0013
```

```
Furniture
```

```
Home furniture products
```

```
Cat0014
```

```
Stationery
```

```
CATEGOR
```

```
-----
```

```
CATEGORY_NAME
```

```
-----
```

```
DESCRIPTION
```

```
-----
```

```
School and office supplies
```

```
SQL> SELECT * FROM Product;
```

```
PRODUCT
```

```
-----
```

```
PRODUCT_NAME
```

```
-----
```

```
CATEGOR      PRICE STOCK_QUANTITY
```

```
-----
```

```
P2001
```

```
Smartphone
```

```
Cat0011      28500          15
```

```
P2002
```

```
Running Shoes
```

```
Cat0012      3499          25
```

```
PRODUCT
```

```
-----
```

```
PRODUCT_NAME
```

```
-----
```

```
CATEGOR      PRICE STOCK_QUANTITY
```

```
-----
```

```
P2003
```

```
Sofa
```

```
Cat0013      42000           8
```

12. CRUD Operations

The Product table supports basic CRUD operations:

- **Create:** Insert a new product record.
- **Read:** Display product records.
- **Update:** Modify the price or other product information.
- **Delete:** Remove a product record.

```
SQL> INSERT INTO Product
```

```
2 (Product_ID, Product_Name, Category_ID, Price, Stock_Quantity)
```

```
3 VALUES
```

```
4 ('P2004', 'Headphones', 'Cat0011', 2499, 20);
```

```
1 row created.
```

```
SQL> UPDATE Product
```

```
2 SET Price = 27000
```

```
3 WHERE Product_Name = 'Smartphone';
```

```
1 row updated.
```

```
SQL> DELETE FROM Product
```

```
2 WHERE Product_Name = 'Headphones';
```

```
1 row deleted.
```

13. Category-wise Product Reports

Display All Products Under Each Category

The products available under each category were displayed along with their price and stock quantity.

Count Number of Products in Each Category

The number of products available in each category was calculated and displayed.

Find Highest-Priced Product in Each Category

The highest-priced product available in each category was identified.

Display Categories Having More Than 5 Products

The categories containing more than five products were identified using the category-wise product count.

Find Average Product Price Category-wise

The average product price for each category was calculated and displayed.

```
SQL> SELECT c.Category_Name,
2         p.Product_Name,
3         p.Price,
4         p.Stock_Quantity
5 FROM Category c
6 JOIN Product p
7 ON c.Category_ID = p.Category_ID;
```

CATEGORY_NAME	PRODUCT_NAME	PRICE	STOCK_QUANTITY
Mobiles	Smartphone	27000	15
Footwear	Running Shoes	3499	25

CATEGORY_NAME	PRODUCT_NAME	PRICE	STOCK_QUANTITY
Furniture	Sofa	42000	8

```
SQL> SELECT c.Category_Name,
2         COUNT(p.Product_ID) AS Product_Count
3 FROM Category c
4 JOIN Product p
5 ON c.Category_ID = p.Category_ID
6 GROUP BY c.Category_Name;
```

CATEGORY_NAME

PRODUCT_COUNT

Mobiles

1

Footwear

1

Furniture

1

```
SQL> SELECT c.Category_Name,
2         p.Product_Name,
3         p.Price
4 FROM Category c
5 JOIN Product p
6 ON c.Category_ID = p.Category_ID
7 WHERE p.Price = (
8         SELECT MAX(p2.Price)
9         FROM Product p2
10        WHERE p2.Category_ID = p.Category_ID
11 );
```

CATEGORY_NAME

PRODUCT_NAME

PRICE

Mobiles

Smartphone

27000

Footwear

Running Shoes

3499

CATEGORY_NAME

PRODUCT_NAME

PRICE

Furniture

Sofa

42000

```
SQL> SELECT c.Category_Name,  
2         AVG(p.Price) AS Average_Price  
3 FROM Category c  
4 JOIN Product p  
5 ON c.Category_ID = p.Category_ID  
6 GROUP BY c.Category_Name;
```

```
CATEGORY_NAME
```

```
-----  
AVERAGE_PRICE
```

```
-----  
Mobiles
```

```
27000
```

```
Footwear
```

```
3499
```

```
Furniture
```

```
42000
```

14. Conclusion

The **Product and Category Management System** was successfully designed and implemented. The Category and Product tables were created, primary key and foreign key relationships were established, and constraints such as **NOT NULL, CHECK, and UNIQUE** were applied. Category and product records were then inserted and used for CRUD operations and category-wise reporting.

This module provides the basic database structure required to manage products and their categories in an e-commerce system.